Complexitylib:
An index of proved statements

19 October 2026

This is a curated index of thirteen results in the Lean library, not an exhaustive API reference. Each entry gives the statement, its model or assumptions, and a link to the formal declaration. Links are pinned to revision `d2c5e581`. The wording and selection remain open to Samuel's editorial direction.

Unless specified otherwise, class statements hold for any type of instances with an injective encoding into finite bit strings; input size is encoded length. Resource bounds and reductions refer to the library's machine definitions. Source citations credit the mathematics; linked modules also record formalization authors. This index was prepared and reviewed with AI assistance; prose is not machine checked.

1 Completeness and verification

1.1 Cook–Levin and its complement

SAT is NP-complete and UNSAT is coNP-complete under tidy polynomial-time many-one reductions, using the library's encoding `encCNF` of CNF formulas. Reduction efficiency is proved, not assumed.[1, Section 2.3]

`Cslib.Complexity.CookLevin.SAT_npComplete`

`Cslib.Complexity.CookLevin.UNSAT_coNPComplete`

1.2 Circuit Value belongs to P

Given a one-output De Morgan circuit and an input of the matching arity, deciding whether its output is true is in P. Instances are well formed by their type; `encCircuitValue` encodes the circuit and input bits. The standard P-completeness theorem under logspace reductions is not yet formalized here. [5]

`Cslib.Complexity.CircuitValue.mem_P`

1.3 The PCP theorem

$$\mathsf{NP} = \mathsf{PCP}(O(\log n), O(1)).$$

The verifier is polynomial-time and nonadaptive, has perfect completeness and soundness at most $1/2$, and uses logarithmically many random bits and constantly many queries. In the Lean equality the bound functions are $\lfloor \log_2 n \rfloor + 1$ (with Lean's natural-number logarithm) and 1; the class definition allows constant multiples of both. It does not assert one query.[4]

`Cslib.Complexity.NP_eq_PCP_log`

2 Space and alternation

2.1 Nondeterministic logspace lies in polynomial time

$$\mathsf{NL} \subseteq \mathsf{P}.$$

NL is defined by native nondeterministic machines with logarithmic space bounded on every computation prefix and acceptance by an accepting path. A polynomial-time decider exists even though the nondeterministic machine's other branches need not halt. This entry does not assert the still-outstanding native $\mathsf{NL} = \mathsf{coNL}$ theorem.[10, Section 8.5]

`Cslib.Complexity.NL_subset_P`

2.2 Savitch's theorem for polynomial space

$$\mathsf{NPSPACE} = \mathsf{PSPACE}.$$

Both classes are machine-defined. Nondeterministic acceptance is by an accepting path, with every path respecting the space bound; the deterministic decider is total. This is the equality of polynomial-space classes, not a statement of the fully parameterized $\mathsf{NSPACE}(s) \subseteq \mathsf{DSPACE}(s^2)$ theorem. [10, Corollary 8.6]

`Cslib.Complexity.NPSPACE_eq_PSPACE`

2.3 Alternating polynomial time equals polynomial space

$$\mathsf{AP} = \mathsf{PSPACE}.$$

AP uses the library's native alternating machines and a polynomial time bound. The equality is with the same machine-defined PSPACE used above; it is not merely an equality of abstract game predicates.[3]

`Cslib.Complexity.AP_eq_PSPACE`

2.4 Interactive proofs lie in polynomial space

$$\mathsf{IP} \subseteq \mathsf{PSPACE}.$$

IP uses a private-coin verifier realized by one fair-coin oracle machine and a polynomial clock valid against every stateful prover. Provers may be randomized and computationally unbounded; completeness is at least $2/3$ and soundness at most $1/3$. The converse $\mathsf{PSPACE} \subseteq \mathsf{IP}$ is not yet formalized here. [1, Chapter 8]

`Cslib.Complexity.IP_subset_PSPACE`

3 Randomness, counting and arithmetic circuits

3.1 Sipser–Gács–Lautemann

$$\mathsf{BPP} \subseteq \Sigma_2^p \cap \Pi_2^p.$$

For the library's bounded-error polynomial-time class, membership has both an existential–universal and a universal–existential characterization with polynomially bounded bit-string witnesses and a polynomial-time predicate. [6, 9]

`Cslib.Complexity.BPP_subset_SigmaP_two_inter_PiP_two`

3.2 Toda's theorem

$$\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}}.$$

Here #P counts witnesses of polynomial-time verifiers. The target class uses a polynomial-time oracle program answering decision problems via bits of one such counting function. The program is fixed before the oracle is chosen and must work for every oracle language agreeing on valid encoded queries; off-code answers are arbitrary. The target class has no separate query-count parameter. The nondeterministic-machine characterization of #P remains separate unfinished work.[11]

`Cslib.Complexity.PH_subset_PWithSharpPOracle`

3.3 Polynomial-size circuits for the determinant

For every commutative ring K and $n \in \mathbb{N}$, the generic $n \times n$ determinant has a division-free arithmetic circuit over K of size at most

$$3n^4 + 4.$$

Consequently its polynomial family belongs to VP over K. The same circuit evaluates determinants over every commutative K-algebra, with matrix entries supplied in row-major order. This is a gate-count assertion, not a certified uniform circuit-generation bound.[2, 12]

`Cslib.Circuits.Arithmetic.exists_circuit_det`

`Cslib.Circuits.Arithmetic.exists_circuit_det_eval`

`Cslib.Circuits.Arithmetic.detFamily_mem_VP`

4 Circuits for sparse counter modifications

Write $W_n = \mathbb{Z}/2^n\mathbb{Z}$ for the n-bit words, represented least-significant bit first, and let $c(x) = x + 1$. For maps on W_n, put

$$D(f, g) = \{x \in W_n : f(x) \neq g(x)\}.$$

Circuit size below counts native De Morgan gates, including constants, comparisons, selection and addition. These are nonuniform existence results: they provide no efficient algorithm for constructing the correction table. The port follows Schlesinger's iterated-functions development; the source credits Moreno and Rivera for the related block characterization of almost-commuting permutations. [7, 8]

4.1 Maps commuting with an edited counter

For any maps $f, h : W_n \to W_n$, if h is injective and $h \circ f = f \circ h$, then there is a circuit computing all n output bits of h with size at most

$$100(n + 1)(|D(f, c)| + 1).$$

There is no assumption that f is a permutation or that a small circuit for f is supplied. The assertion includes $n = 0$.

`Cslib.Circuits.Boolean.Iteration.sparse_counter_centralizer`

4.2 Every fixed signed power

For every permutation f of W_n and every integer T, a circuit computes f^T with size at most

$$100(n + 1)(|D(f, c)| + 1).$$

The size bound is independent of the magnitude and sign of T. The quantifiers are "for every T, there exists a circuit": T is fixed, not an additional input to a single circuit.

`Cslib.Circuits.Boolean.Iteration.sparse_counter_power`

4.3 Transfer through supplied coordinates

Let f, g, e be permutations of W_n such that $e(g(x)) = e(x) + 1$ for every x. Suppose circuits E and E^{-1} compute e and its inverse, respectively. For every $T \in \mathbb{Z}$, there is a circuit for f^T with size at most

$$|E| + |E^{-1}| + 100(n + 1)(|D(f, g)| + 1).$$

Both coordinate conversions are charged. The theorem does not assert that an arbitrary cycle has cheap coordinates or complete the general iterated-function circuit-bound program.

`Cslib.Circuits.Boolean.Iteration.sparse_coordinate_power`

References

[1] Sanjeev Arora and Boaz Barak. *Computational Complexity - A Modern Approach*. Cambridge University Press, 2009.

[2] Richard S. Bird. A simple division-free algorithm for computing determinants. *Information Processing Letters*, 111(21–22):1072–1074, 2011. doi: 10.1016/j.ipl.2011.08.006.

[3] Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. Alternation. *Journal of the ACM*, 28(1):114–133, 1981. doi: 10.1145/322234.322243.

[4] Irit Dinur. The PCP theorem by gap amplification. *Journal of the ACM*, 54(3):12, 2007. doi: 10.1145/1236457.1236459.

[5] Richard E. Ladner. The circuit value problem is log space complete for P. *ACM SIGACT News*, 7(1):18–20, 1975. doi: 10.1145/990518.990519.

[6] Clemens Lautemann. BPP and the polynomial hierarchy. *Information Processing Letters*, 17 (4):215–217, 1983. doi: 10.1016/0020-0190(83)90044-3.

[7] Rutilo Moreno and Luis Manuel Rivera. Blocks in cycles and k-commuting permutations. *SpringerPlus*, 5:1949, 2016. doi: 10.1186/s40064-016-3638-7. URL https://doi.org/10.1186/s40064-016-3638-7. Article 1949.

[8] Samuel Schlesinger. Square-root circuit bounds for iterated functions, 2026. URL https://github.com/SamuelSchlesinger/iterating-functions/tree/c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b/lean. Source of ported Lean proofs, revision c4cfa198a241b3d2cc3bfa5f744a68a66f49be8b.

[9] Michael Sipser. A complexity theoretic approach to randomness. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing*, pages 330–335, 1983. doi: 10.1145/800061.808762. URL https://doi.org/10.1145/800061.808762.

[10] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2013.

[11] Seinosuke Toda. PP is as hard as the polynomial-time hierarchy. *SIAM Journal on Computing*, 20(5):865–877, 1991. doi: 10.1137/0220053. URL https://doi.org/10.1137/0220053.

[12] Leslie G. Valiant. Completeness classes in algebra. In *Proceedings of the Eleventh Annual ACM Symposium on Theory of Computing*, STOC '79, pages 249–261. ACM, 1979. doi: 10.1145/800135.804419.